

Parameter_Injection-crytohack

cryptograph

Analisis

deskripsi chall=

You're in a position to not only intercept Alice and Bob's DH key exchange, but also rewrite their messages. Think about how you can play with the DH equation that they calculate, and therefore sidestep the need to crack any discrete logarithm problem.

Use the script from "Deriving Symmetric Keys" to decrypt the flag once you've recovered the shared secret.

Connect at socket.crytohack.org 13371

saat kita nc socket.crytohack.org 13371 maka akan muncul

```
(siber@ITNSA)~[/mnt/d/py_repin]
$ nc socket.crytohack.org 13371
Intercepted from Alice: {"p": "0xffffffffffffffffc90fdaa22168c234c4c6628b80dc1cd129024e088a67cc74020bbea63b139b22514a087
98e3404ddef9519b3cd3a431b302b0a6df25f14374fe1356d6d51c245e485b576625e7ec6f44c42e9a637ed6b0bfff5cb6f406b7edee386bfb5a899fa
5ae9f24117c4b1fe649286651ece45b3dc2007cb8a163bf0598da48361c55d39a69163fa8fd24cf5f83655d23dca3ad961c62f356208552bb9ed5290
77096966d670c354e4abc9804f1746c08ca237327ffffffffffffffff", "g": "0x02", "A": "0xa28d8c52b1e8b0a54dda19d2804cf514c1c4a39
bc4a024e6cbcd3f0fb2b1c864226408ac85f049951feb10937572cb331ab0b01f55c6a5e09dadef20b460219084d71fca2b6afaeed50cf8b49003d6e
35f78fc50117804fcd7fca9fce3d44f0ef431a1ea36acb3480c001462465bdd80341d5ba503ee71453ccbd06dd5e4c37c6b057de55057c4e0dde99c7
1602cd3d48cb95d49ea724ba9d1352ab344be4b6803eb6ff53311a6c08b1daf987726dc07d757c5535b3013f2081f323a6edbd2c6"}
Send to Bob: |
```

disini kita jadi orang yang berada di tengah tengah antara Alice dan Bob yang sedang berkomunikasi, disini kita bisa mencegat pesan dari Alice sebelum kita kirimkan ke bob. setelah kita kirim pesan dari Alice yang berisi pesan p, g, dan A ke Bob, maka Bob akan merespon dengan memberikan B setelah kita mengirim B ke Alice maka Alice akan merespon dengan memberikan iv dan ct_flag

```

(siber@ITNSA)-[ /mnt/d/py_repin ]
$ nc socket.cryptohack.org 13371
Intercepted from Alice: {"p": "0xffffffffffffffff90fdaa22168c234c4c6628b80dc1cd129024e088a67cc74020bbea63b139b22514a087
98e3404ddef9519b3cd3a431b302b0a6df25f14374fe1356d6d51c245e485b576625e7ec6f44c42e9a637ed6b0bff5cb6f406b7edee386bfb5a899fa
5ae9f24117c4b1fe649286651ece45b3dc2007cb8a163bf0598da48361c55d39a69163fa8fd24cf5f83655d23dca3ad961c62f356208552bb9ed5290
77096966d670c354e4abc9804f1746c08ca237327ffffffffffffffff", "g": "0x02", "A": "0xa28d8c52b1e8b0a54dda19d2804cf514c1c4a39
bc4a024e6cbcd3f0fb2b1c864226408ac85f049951feb10937572cb331ab0b01f55c6a5e09dadedf20b460219084d71fca2b6afaeed50cf8b49003d6e
35f78fc50117804fcd7fca9fce3d44f0ef431a1ea36acb3480c001462465bdd80341d5ba503ee71453ccbd06dd5e4c37c6b057de55057c4e0dde99c7
1602cd3d48cb95d49ea724ba9d1352ab344be4b6803eb6ff53311a6c08b1daf987726dc07d757c5535b3013f2081f323a6edbd2c6"}
Send to Bob: {"p": "0xffffffffffffffff90fdaa22168c234c4c6628b80dc1cd129024e088a67cc74020bbea63b139b22514a08798e3404ddef
9519b3cd3a431b302b0a6df25f14374fe1356d6d51c245e485b576625e7ec6f44c42e9a637ed6b0bff5cb6f406b7edee386bfb5a899fa5ae9f24117c
4b1fe649286651ece45b3dc2007cb8a163bf0598da48361c55d39a69163fa8fd24cf5f83655d23dca3ad961c62f356208552bb9ed529077096966d67
0c354e4abc9804f1746c08ca237327ffffffffffffffff", "g": "0x02", "A": "0xa28d8c52b1e8b0a54dda19d2804cf514c1c4a39bc4a024e6cb
ed3f0fb2b1c864226408ac85f049951feb10937572cb331ab0b01f55c6a5e09dadedf20b460219084d71fca2b6afaeed50cf8b49003d6e35f78fc5011
7804fcd7fca9fce3d44f0ef431a1ea36acb3480c001462465bdd80341d5ba503ee71453ccbd06dd5e4c37c6b057de55057c4e0dde99c71602cd3d48c
b95d49ea724ba9d1352ab344be4b6803eb6ff53311a6c08b1daf987726dc07d757c5535b3013f2081f323a6edbd2c6"}
Intercepted from Bob: {"B": "0x2665a31d22d3c101a3f8e1f87f01eb5d746e096f675fe11fcc19033cbfddad1db60c217d680e111fd44846872
5054ee1007143caf09fa325570d34651763394bb2307a6048d42e9ad6d0f6b197b89494b247ca1715cdec2908622a7015f3b1db0cdf3b7767b33f1aa
5244cf15a704d3c9a26166a139cba01666b417a2a356ecf0e234f6d1c0acab9ac57230754c44b33dd90d99c740ff3c754fe383b57ddc8605339c748
c202c9b30bc4128150c742d6db548e5e6a3236b4a513b53b11d4f74"}
Send to Alice: {"B": "0x2665a31d22d3c101a3f8e1f87f01eb5d746e096f675fe11fcc19033cbfddad1db60c217d680e111fd448468725054ee1
007143caf09fa325570d34651763394bb2307a6048d42e9ad6d0f6b197b89494b247ca1715cdec2908622a7015f3b1db0cdf3b7767b33f1aa5244cf
15a704d3c9a26166a139cba01666b417a2a356ecf0e234f6d1c0acab9ac57230754c44b33dd90d99c740ff3c754fe383b57ddc8605339c748c202c9b
30bc4128150c742d6db548e5e6a3236b4a513b53b11d4f74"}
Intercepted from Alice: {"iv": "666935ab1a3d627f362ab0bd0f6c7fd8", "encrypted_flag": "4f4a20df56ff30e1cfc58d3471f1e2e17b3ca4803df7118cd3b22
e04d27197db"}

```

Solusi

di karenakan kita bisa memanipulasi text sebelum mengirimnya ke Alice dan Bob maka kita bisa mendapatkan S tanpa tau a maupun b dari Alice dan Bob, sebelum mengirim pesan Alice ke Bob kita rubah dulu pesan nya menjadi

{"p":

"0xffffffffffffffff90fdaa22168c234c4c6628b80dc1cd129024e088a67cc74020bbea63b139b22514a08798e3404ddef9519b3cd3a431b302b0a6df25f14374fe1356d6d51c245e485b576625e7ec6f44c42e9a637ed6b0bff5cb6f406b7edee386bfb5a899fa5ae9f24117c4b1fe649286651ece45b3dc2007cb8a163bf0598da48361c55d39a69163fa8fd24cf5f83655d23dca3ad961c62f356208552bb9ed529077096966d670c354e4abc9804f1746c08ca237327ffffffffffffffff", "g": "0x02", "A": "0x01"}

lalu setelah Bob mengirim B,kita ubah dulu sebelum mengirimkan nya ke Alice menjadi

{"B": "0x01"}

setelah Alice memberi respon

**{"iv": "666935ab1a3d627f362ab0bd0f6c7fd8",
"encrypted_flag": "4f4a20df56ff30e1cfc58d3471f1e2e17b3ca4803df7118cd3b22e04d27197db"}**

maka kita tinggal mendekripsikan ct_flag karena kita tahu bahwa S = 0x01,kenapa kita bisa tau ini?karena tadi kita sudah memanipulasi A dan B menjadi 0x01 yang dimana nilai S otomatis menjadi 0x01 jadi kita tinggal mendekripsi flag dengan solver ini

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import hashlib

def is_pkcs7_padded(message):
    padding = message[-message[-1]:]
    return all(padding[i] == len(padding) for i in range(0,
len(padding)))

def decrypt_flag(shared_secret: int, iv: str, ciphertext: str):
    # Derive AES key from shared secret
    sha1 = hashlib.sha1()
    sha1.update(str(shared_secret).encode('ascii'))
    key = sha1.digest()[:16]
    # Decrypt flag
    ciphertext = bytes.fromhex(ciphertext)
    iv = bytes.fromhex(iv)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    plaintext = cipher.decrypt(ciphertext)

    if is_pkcs7_padded(plaintext):
        return unpad(plaintext, 16).decode('ascii')
    else:
        return plaintext.decode('ascii')

shared_secret = 1
iv = 'c5fffc184c1338ce99a0137123ccaf1c'
ciphertext =
'01d84781690f7014b91933eb7c9dde3d23fda9f570fc65a0bc455767ca033a58'

print(decrypt_flag(shared_secret, iv, ciphertext))
```